

Generics in Java

Aspekt	Beschreibung
Definition	Generics ermöglichen die Verwendung parametrisierter Typen in Klassen, Interfaces und Methoden, um Typsicherheit zur Compile-Zeit zu garantieren.
Zielsetzung	Vermeidung von Typ-Casts. Früherkennung von Typfehlern. Wiederverwendbarkeit. saubere API-Definition
Syntax (grundlegend)	Ein Typparameter wird in spitzen Klammern <code><></code> definiert, z. B. <code>class Box<T></code> oder <code>List<String></code> .
Typische Typ-Parameter	<code>T</code> : Type. <code>E</code> : Element. <code>K</code> : Key. <code>V</code> : Value. <code>R</code> : Return (Konvention, keine Einschränkung)
Type Erasure	Der Compiler ersetzt alle generischen Typen durch ihren Bound (oder <code>Object</code> , wenn kein Bound vorhanden ist). Dies geschieht zur Laufzeit.
Typsicherheit	Generics verhindern <code>ClassCastException</code> zur Laufzeit durch frühzeitige Prüfung beim Kompilieren.
Roh-Typen (Raw Types)	Verwendung eines generischen Typs ohne Typ-Parameter (z. B. <code>List</code> statt <code>List<String></code>). Führt zu Warnungen und potenziell unsicherem Code.
Wildcards (?)	Platzhalter für einen unbekannten Typ: <code>? extends T</code> – obere Schranke (nur lesen). <code>? super T</code> – untere Schranke (nur schreiben)
Ungebundene Wildcards	<code><?></code> – erlaubt beliebige Typen, nützlich z. B. für Methoden mit rein lesendem Zugriff auf eine Collection
Kovarianz (extends)	Typkompatibilität bei nur-lesendem Zugriff – erlaubt z. B. <code>List<? extends Number></code> für <code>List<Integer></code> und <code>List<Double></code>
Kontravarianz (super)	Typkompatibilität bei schreibendem Zugriff – erlaubt z. B. <code>List<? super Integer></code> für <code>List<Number></code> oder <code>List<Object></code>
Bounded Type Parameters	Einschränkung eines Typparameters z. B. <code>T extends Number</code> oder <code>T extends Comparable<T></code>
Mehrfache Bounds	Kombination von mehreren Schranken, z. B. <code>T extends Number & Comparable<T></code>
Generische Methoden	Methoden können unabhängig vom Klassentyp eigene Typparameter definieren, z. B. <code><T> T identity(T input)</code>
Generics in Interfaces	Interfaces können ebenfalls Typ-Parameter besitzen, z. B. <code>interface Repository<T></code>

Aspekt	Beschreibung
Generics und Vererbung	Subklassen müssen generischen Parameter spezifizieren oder selbst generisch sein. Kein automatisches Subtyping: <code>List<Integer> ≠ List<Number></code>
Generics vs. Arrays	Arrays sind kovariant (unsicher), Generics sind invariant (sicherer). Man kann keine generischen Arrays direkt erzeugen (<code>new T[]</code>).
Einschränkungen	Keine direkten Instanzen generischer Typen (<code>new T()</code>). Kein <code>instanceof</code> mit generischem Typ. Kein <code>static</code> -Kontext mit <code>T</code>
Namenskollisionen	Innerhalb verschachtelter Klassen können gleichnamige Typ-Parameter zu Konflikten führen – klare Namenswahl ist wichtig.
Typinferenz	Der Compiler kann den Typ oft automatisch ableiten (z. B. <code>var list = List.of("A", "B");</code>)
PECS-Regel	<i>Producer Extends, Consumer Super</i> – bei Lesenden: <code>extends</code> , bei Schreibenden: <code>super</code>
Generics und Überladung	Methodenüberladung nur begrenzt möglich, da durch Type Erasure manche Kombinationen nicht unterscheidbar sind.
API-Design mit Generics	Generics ermöglichen flexible, wiederverwendbare und sichere Bibliotheken. Besonders wichtig für Collections, Utilities, Repositories.
Best Practices	Keine Raw Types. Keine generischen Arrays. Verwendung klarer Typparameter. Wildcards bewusst einsetzen. Kollisionen vermeiden
Typische Fehlerquellen	Verwechslung von <code>extends</code> und <code>super</code> . Einsatz generischer Arrays. Annahme falscher Subtyp-Kompatibilität